

31/08/2026

Linked List Level - 1 : Foundation

① Linked List Traversal :

Problem: Given $[10 | \cdot] \rightarrow [20 | \cdot] \rightarrow [30 | \cdot] \rightarrow [40 | \text{None}]$

↑
Head.

o/p: 10, 20 30 40

In Array: `nums = [10, 20, 30, 40]`

for `i` in range(`len(nums)`):

`print(nums[i])`

} here we use index
But
In linked list we do

Instead: `head` $\rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$

Recognition words: Traverse linked list, print all nodes
visit every node Display linked list, Iterate through
linked list

Python Node:

class Node:

`def __init__(self, data):`

`self.data = data`

`self.next = None`

} Node Structure

Creating nodes:

`first = Node(10)`

`second = Node(20)`

`third = Node(30)`

`fourth = Node(40)`

Connect:

`first.next = second`

`second.next = third`

`third.next = fourth`

First

So: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$

Set:

`head = first`

Algorithm:

1. `current = head`

2. While `current` is not `None`:

`process current.data`

`current = current.next`

3. Stop when `current` becomes `None`

Traversal Template:

```
def print_list(head):  
    current = head  
    while current is not None:  
        print(current.data)  
        current = current.next
```

T.C: $O(n)$
S.C: $O(1)$

Array Traversal vs Linked List Traversal

Array:

```
for i in range(len(nums)):  
    print(nums[i])
```

Think: Index

Linked List:

current = head

while current:

print(current.data)

current = current.next

Think:

Link / Reference

Core Idea:

Head



current



Process



current = current.next



None → stop.

Golden Rule:

Linked List:

→ Start at Head

→ Follow Next

→ Stop at None

② Search an Element in Linked List

Problem: Find the target element in the given linked list. $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$
target: 30

Core Idea:

Current.data == target?

↓

YES → Found

No → Move next

Algorithm:

Current = head

while current:

if Current.data == target:

return True

Current = Current.next

return False

Traversal
↑

Comparison

Python Template:

```
def search(head, target):
```

```
    current = head
```

```
    while current:
```

```
        if current.data == target:
```

```
            return True
```

```
        current = current.next
```

```
    return False
```

T.C: $O(n)$

S.C: $O(1)$

③ Insert a Node

Problem: In an existing list $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{None}$

Inserting 15 in between of $10 \rightarrow 20$.

Expected: $10 \rightarrow 15 \rightarrow 20 \rightarrow 30 \rightarrow \text{None}$

Algorithm:

create new node

↓

find the node after which insertion should happen

↓

connect new node to next node

↓

connect current node to new node

In code:

`new_node.next = current.next`

`current.next = new_node`

Python Code:

class Node:

`def __init__(self, data):`

`self.data = data`

`self.next = None`

`def insert_after (head, target, value):`

`current = head`

`while current:`

`if current.data == target:`

`new_node = Node (value)`

`new_node.next = current.next`

`current.next = new_node`

`return head`

`current = current.next`

`return head`

Insert at Beginning

Existing: $20 \rightarrow 30 \rightarrow 40$
 ↑
 head

Insert 10 at a beginning

Code:

`new_node = Node(10)`

`new_node.next = head`

`head = new_node`

Insert at End:

Existing: $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{None}$, Insert 40 at the end.

Code: `current.next = new_node`

④ Delete a node:

problem: Given $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$

delete 20 then $10 \rightarrow 30 \rightarrow 40 \rightarrow \text{None}$.

Algorithm:

check if list is empty

↓

check if head itself is target

(move head to head.next)

otherwise:

Traverse until:

`current.next.data == target`

Bypass target:

`current.next = current.next.next`

Return head.

Notice:

To delete a node, usually we need access to the node before it.

`previous.next = target.next`

Python code:

```
def delete_node(head, target):
```

```
    # empty list
```

```
    if head is None:
```

```
        return None
```

```
    # target is the head
```

```
    if head.data == target:
```

```
        return head.next
```

```
    current = head
```

```
    while current.next:
```

```
        if current.next.data == target:
```

```
            current = current.next
```

```
    return head
```

T.C: $O(n)$

S.C: $O(1)$
